PROJECT REPORT

Cancer Classification and Prediction Using Neural Networks

Deep learning for breast-tumour classification on the Breast Cancer Wisconsin (Diagnostic) dataset

Vaidehi Aggarwal
B.Tech, Computer Science Engineering — 5th Semester
Vivekananda Institute of Professional Studies, GGSIPU
July 2026

EXECUTIVE SUMMARY

A neural network that classifies tumours with 96.49% test accuracy

A Sequential neural network (30 → 16 → 8 → output) was trained on quantitative cell-nucleus measurements from the Breast Cancer Wisconsin (Diagnostic) dataset to classify tumours as benign or malignant. Trained for 10 epochs with the Adam optimiser, the model comfortably exceeded the 90% accuracy target set for the project and generalised well from training to unseen data.

The system was further validated by predicting on new, unseen patient measurements — correctly identifying tumour status with high confidence — confirming its viability as a decision-support proof of concept.

96.49% Test-set accuracy

98.29% Training accuracy

569 Patient records

30 Input features

INTRODUCTION

Why automate tumour classification?

Conventional diagnosis depends on a pathologist examining tissue samples — a process that is slow, demands scarce specialist expertise, and carries inherent inter-observer variability. As case volumes rise, these constraints become a bottleneck in the diagnostic pathway.

1

Accuracy above 90%

Distinguish benign from malignant tumours reliably enough to support clinical decisions.

2

Seconds, not hours

Return a result far faster than manual slide review.

3

Consistent criteria

Apply identical logic to every case, removing observer-to-observer variability.

4

Minimal false negatives

Keep missed malignancies — the costliest error — to a minimum.

LITERATURE REVIEW

Where this project sits in prior work

Machine learning in healthcare

A decade of rapid progress in deep learning has produced diagnostic systems rivalling specialist clinicians on narrow tasks such as skin-lesion classification and diabetic-retinopathy screening. Performance tracks data quality and quantity more than model complexity.

Neural networks & backpropagation

Layers of interconnected units compute weighted sums through non-linear activations; backpropagation computes gradients and an optimiser updates weights. Adam — adaptive per-parameter learning rates — is now a standard, low-tuning choice.

Prior work on this dataset

Used as a benchmark since the early 1990s, when introduced alongside a linear-programming diagnosis approach. SVMs, logistic regression, random forests, and neural networks have all reported 94–98% accuracy — suggesting the classes are well separated and preprocessing matters more than classifier choice.

The role of feature scaling

Features span very different ranges — mean area in the thousands, mean smoothness under 0.2. Without standardisation, gradient descent is dominated by large-magnitude features. Rescaling to zero mean and unit variance is a requirement, not a refinement.

What the project set out to do

OBJECTIVES

- Classify breast tumours as benign or malignant
- Achieve test-set accuracy above 90%
- Implement a complete preprocessing pipeline, incl. standardisation
- Evaluate using accuracy, loss curves, and error analysis
- Build a working prediction system for new patient data
- Demonstrate that the model generalises rather than overfits

SCOPE

Covers the full ML workflow — loading data, exploratory analysis, quality checks, train/test split, standardisation, network design and training, evaluation, and prediction on new input. Deliberately limited to binary classification on the Wisconsin dataset; multiclass subtyping, imaging integration, and clinical deployment are future work.

DELIVERABLES

- **Source code:** Jupyter Notebook (.ipynb) and Python files
- **Documentation:** This project report
- **Evidence:** Screenshots of execution, training output, results
- **Model:** Trained Keras Sequential network

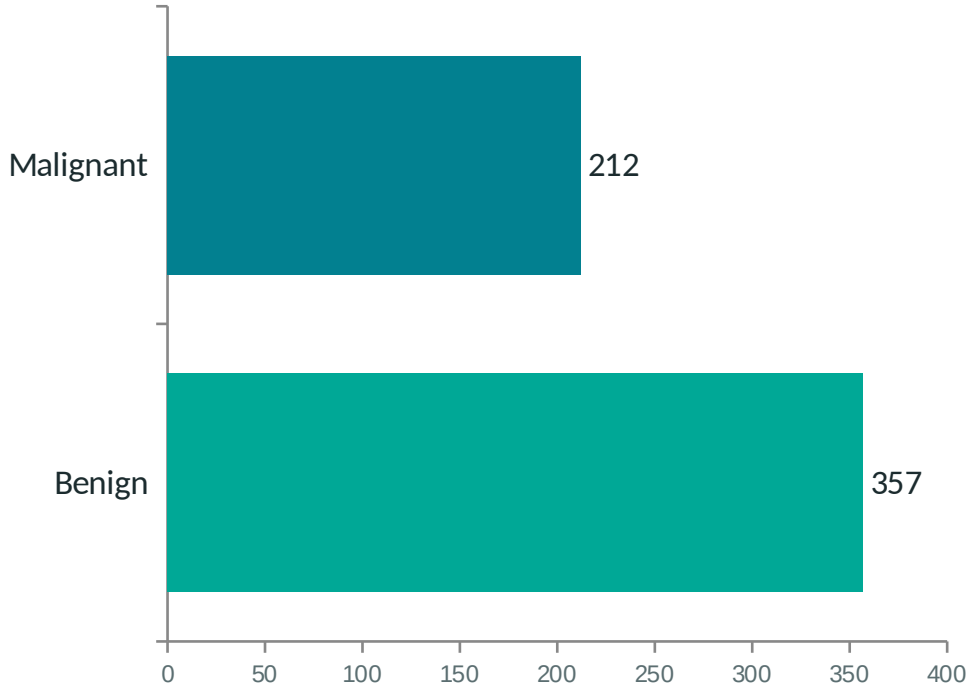
DATASET DESCRIPTION

Breast Cancer Wisconsin (Diagnostic) dataset

569 patient records, 30 numeric features derived from ten cell-nucleus measurements (radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, fractal dimension) — each recorded as mean, standard error, and worst value. An explicit null check returned zero missing values across every column, so the full sample was available for training and testing.

569	30	0
Total samples	Numeric features	Missing values

CLASS DISTRIBUTION (569 SAMPLES)



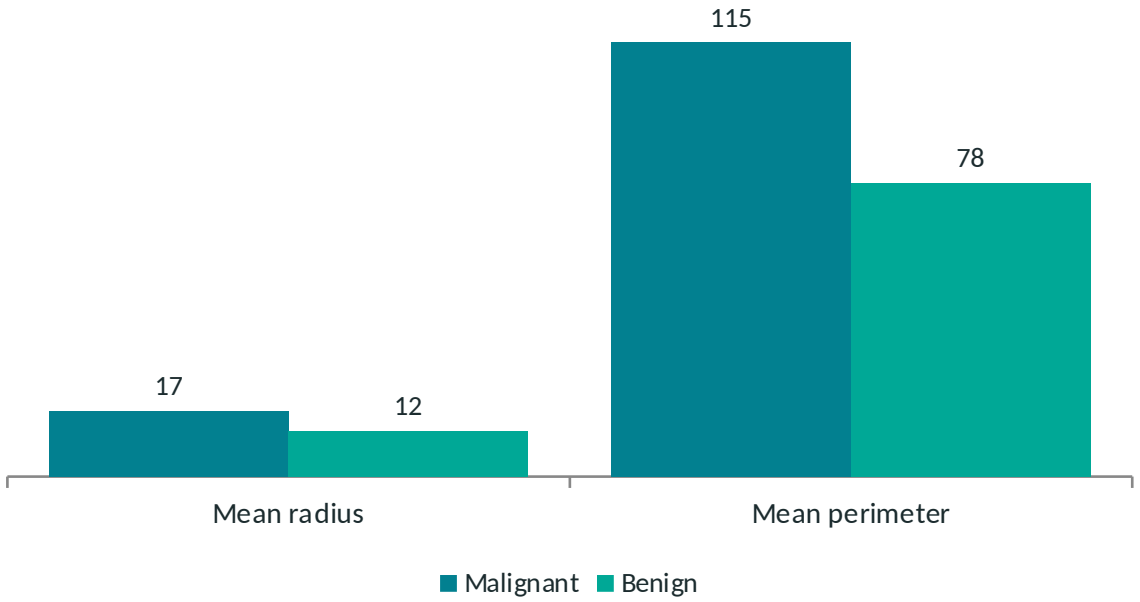
Benign: 357 (63%) • Malignant: 212 (37%) — a mild imbalance that trains well without resampling.

DATASET ANALYSIS

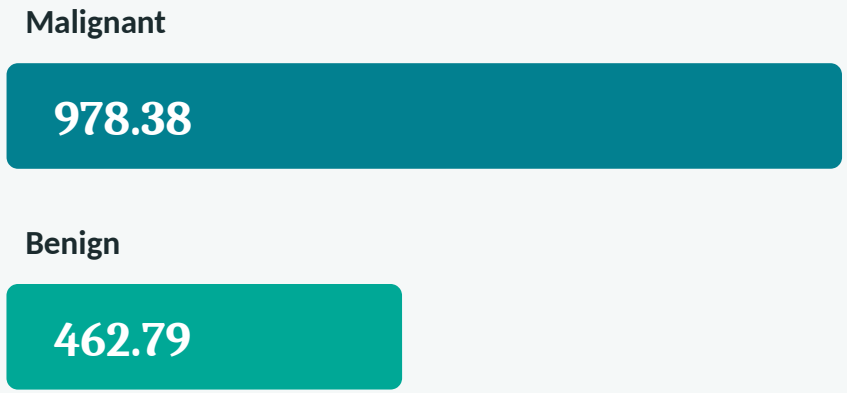
Classes are clearly separable in feature space

Grouping the data by label and computing feature means reveals a wide gap between the two classes. Malignant tumours show substantially higher mean radius, perimeter, and area — this separation is what makes the classification problem tractable for a neural network.

MEAN RADIUS & PERIMETER, BY CLASS



MEAN AREA, BY CLASS



Malignant tumours average roughly 2.1× the mean area of benign tumours — a difference too large for the shared axis above.

NEURAL NETWORK DESIGN

A funnel-shaped feedforward network

Layer sizes narrow progressively from 30 → 16 → 8, compressing the input into an increasingly compact representation that retains diagnostic signal while discarding noise.

30	16	8	1
Input	Hidden 1	Hidden 2	Output
features	ReLU	ReLU	Benign / Malignant

Loss: Cross-entropy **Optimiser:** Adam **Epochs:** 10 **Val. split:** 10%

IMPLEMENTATION & METHODOLOGY

From raw data to a trained model

Python 3.12 • TensorFlow / Keras • NumPy & Pandas • scikit-learn • Matplotlib • Google Colab

1 Load & inspect

Load via scikit-learn into a Pandas dataframe; confirm 569 rows × 31 columns, no nulls.

2 Train / test split

80/20 split with a fixed random state → 455 training, 114 test samples.

3 Standardise

Fit StandardScaler on training data only, then transform both sets — avoids leakage.

4 Train the network

Fit for 10 epochs, Adam optimiser, 10% validation split, monitoring accuracy.

5 Evaluate

Score on the held-out test set; plot accuracy and loss curves.

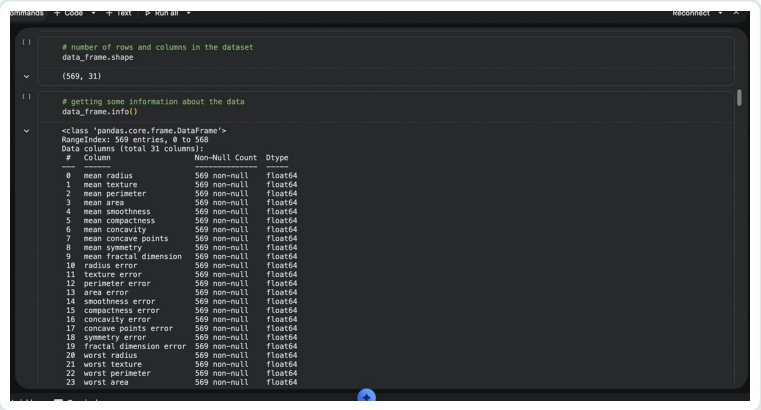
6 Predict

Feed new patient measurements through the same fitted scaler and model.

NOTEBOOK EVIDENCE

Data preparation, straight from the notebook

Screenshots from the executed Google Colab notebook, confirming the dataset shape, the missing-value check, and the train/test split described in the methodology.



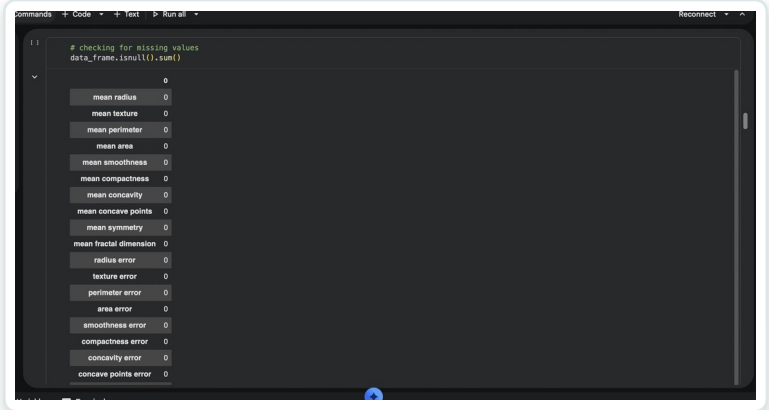
```
# number of rows and columns in the dataset
data_frame.shape
```

```
(569, 31)
```

```
# getting some information about the data
data_frame.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 569 entries, 0 to 568
Data columns (total 31 columns):
 #   Column                   Non-Null Count  Dtype
---  ------                   --------------  -----
 0   mean radius              569 non-null    float64
 1   mean texture             569 non-null    float64
 2   mean perimeter           569 non-null    float64
 3   mean area                569 non-null    float64
 4   mean smoothness          569 non-null    float64
 5   mean compactness         569 non-null    float64
 6   mean concavity           569 non-null    float64
 7   mean concave points      569 non-null    float64
 8   mean symmetry            569 non-null    float64
 9   mean fractal dimension   569 non-null    float64
 10  radius error             569 non-null    float64
 11  texture error            569 non-null    float64
 12  perimeter error          569 non-null    float64
 13  area error               569 non-null    float64
 14  smoothness error         569 non-null    float64
 15  compactness error        569 non-null    float64
 16  concavity error          569 non-null    float64
 17  concave points error     569 non-null    float64
 18  symmetry error           569 non-null    float64
 19  fractal dimension error  569 non-null    float64
 20  worst radius             569 non-null    float64
 21  worst texture            569 non-null    float64
 22  worst perimeter          569 non-null    float64
 23  worst area               569 non-null    float64
```

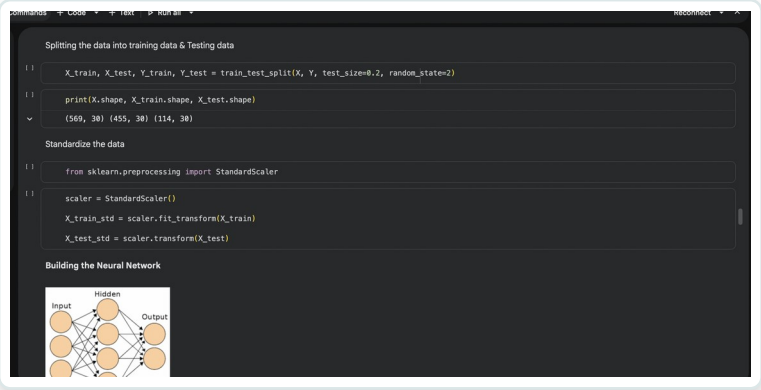
Fig. — dataset shape & dtypes (569 × 31)



```
# checking for missing values
data_frame.isnull().sum()
```

	0
mean radius	0
mean texture	0
mean perimeter	0
mean area	0
mean smoothness	0
mean compactness	0
mean concavity	0
mean concave points	0
mean symmetry	0
mean fractal dimension	0
radius error	0
texture error	0
perimeter error	0
area error	0
smoothness error	0
compactness error	0
concavity error	0
concave points error	0

Fig. — null check, zero missing values



Splitting the data into training data & Testing data

```
X_train, X_test, Y_train, Y_test = train_test_split(X, Y, test_size=0.2, random_state=2)
```

```
print(X.shape, X_train.shape, X_test.shape)
```

```
(569, 30) (455, 30) (114, 30)
```

Standardize the data

```
from sklearn.preprocessing import StandardScaler
```

```
scaler = StandardScaler()
X_train_std = scaler.fit_transform(X_train)
X_test_std = scaler.transform(X_test)
```

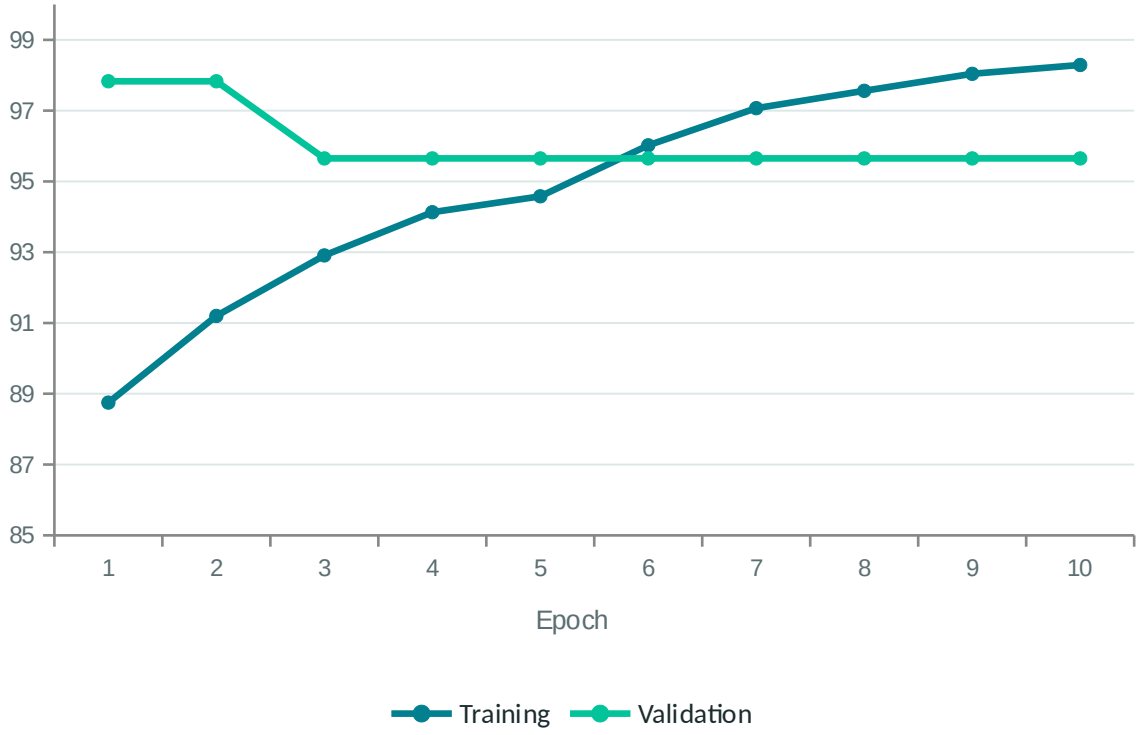
Building the Neural Network

Fig. — 80/20 train/test split & standardisation

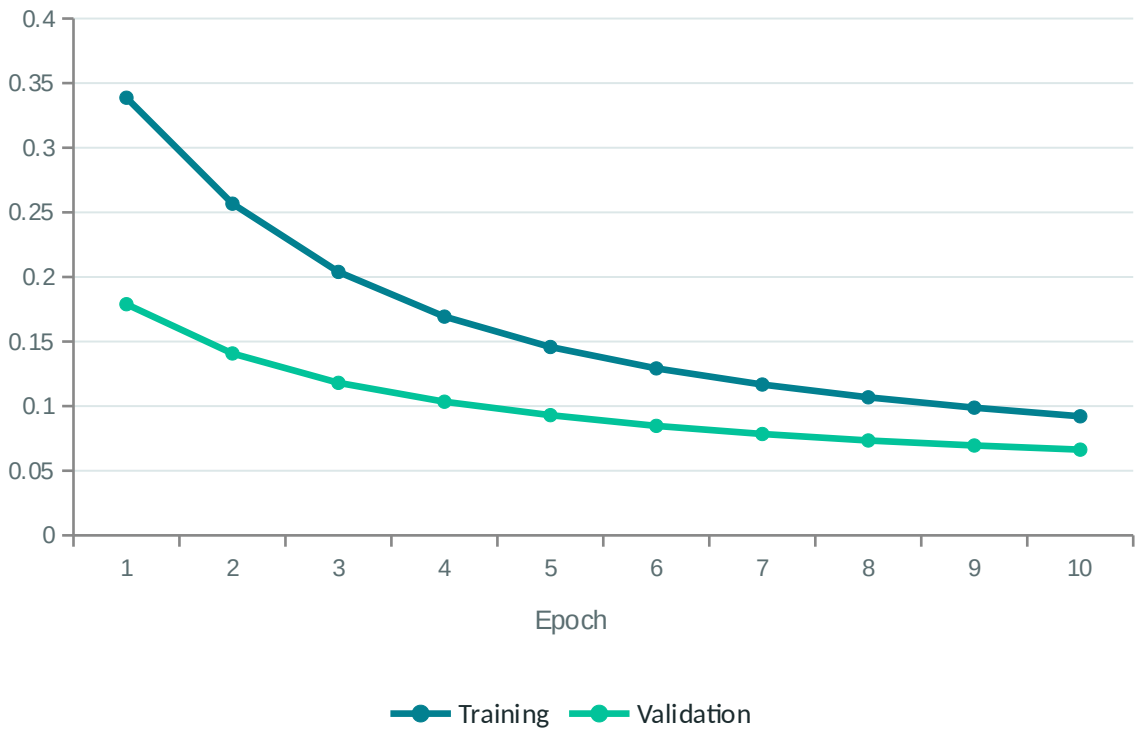
MODEL TRAINING

Accuracy climbs, loss falls, over 10 epochs

ACCURACY (%)



LOSS



Validation loss declines steadily throughout — no upward turn — indicating the model is not overfitting within these 10 epochs.

NOTEBOOK EVIDENCE

Training run, straight from the notebook

The raw epoch-by-epoch Keras output and the matplotlib accuracy/loss plots generated directly from the training history object.

```
metrics=['accuracy'])

# training the Meural Network

history = model.fit(X_train_std, Y_train, validation_split=0.1, epochs=10)

Epoch 1/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 2s 38ms/step - accuracy: 0.8875 - loss: 0.3387 - val_accuracy: 0.9783 - val_loss: 0.1789
Epoch 2/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9120 - loss: 0.2567 - val_accuracy: 0.9783 - val_loss: 0.1408
Epoch 3/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 15ms/step - accuracy: 0.9291 - loss: 0.2039 - val_accuracy: 0.9565 - val_loss: 0.1180
Epoch 4/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - accuracy: 0.9413 - loss: 0.1693 - val_accuracy: 0.9565 - val_loss: 0.1034
Epoch 5/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9560 - loss: 0.1458 - val_accuracy: 0.9565 - val_loss: 0.0930
Epoch 6/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 28ms/step - accuracy: 0.9682 - loss: 0.1292 - val_accuracy: 0.9565 - val_loss: 0.0847
Epoch 7/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 21ms/step - accuracy: 0.9707 - loss: 0.1167 - val_accuracy: 0.9565 - val_loss: 0.0784
Epoch 8/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 18ms/step - accuracy: 0.9756 - loss: 0.1068 - val_accuracy: 0.9565 - val_loss: 0.0734
Epoch 9/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - accuracy: 0.9804 - loss: 0.0988 - val_accuracy: 0.9565 - val_loss: 0.0695
Epoch 10/10
13/13 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - accuracy: 0.9829 - loss: 0.0921 - val_accuracy: 0.9565 - val_loss: 0.0663
```

Visualizing accuracy and loss

```
plt.plot(history.history['accuracy'])
plt.plot(history.history['val_accuracy'])

plt.title('model accuracy')
plt.ylabel('accuracy')
```

Fig. — epoch-by-epoch training log (10 epochs)

Fig. — training vs. validation accuracy

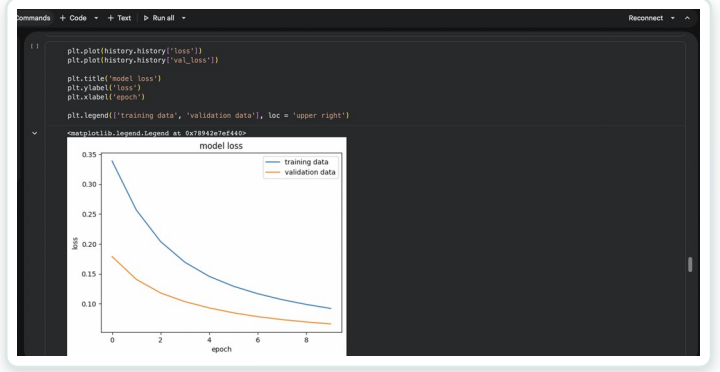


Fig. — training vs. validation loss

RESULTS & PERFORMANCE

Test-set performance

96.49%

TEST ACCURACY

Evaluating on the 114 held-out test samples gives 96.49% accuracy with a test loss of 0.1059 — roughly 110 of 114 samples classified correctly, and a clear margin above the 90% project target. The gap between training (98.29%) and test accuracy is under two points, indicating the network learned the underlying relationship rather than memorising training records.

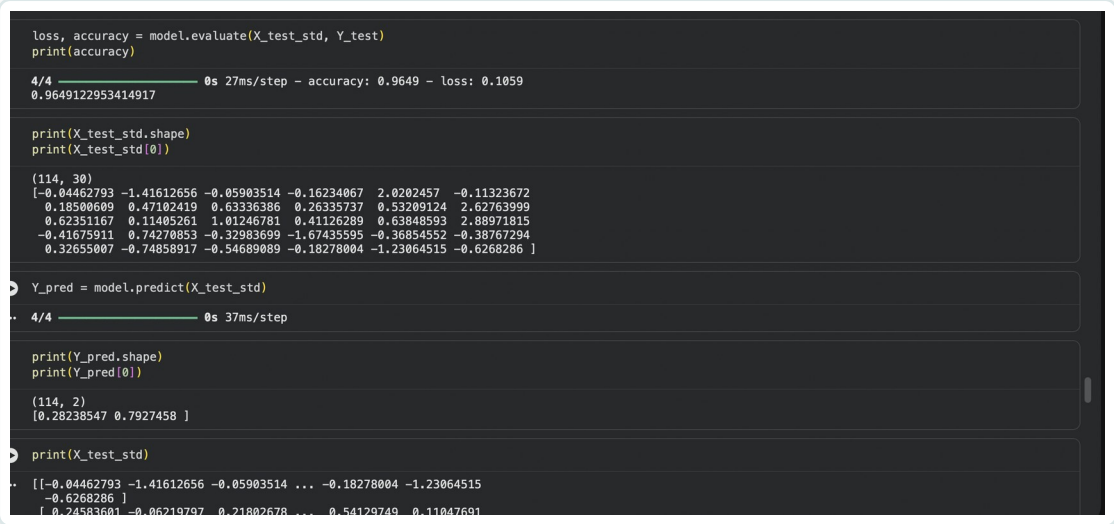
READ THE NUMBER CAREFULLY

- With only 114 test samples, each misclassification moves accuracy by ~0.88 points — a different random split would likely land between 94% and 98%.
- Overall accuracy is not the most important metric in diagnostics: a false negative (malignant read as benign) carries far greater consequence than a false positive.
- Sensitivity, specificity, and a confusion matrix were not yet computed — identified as necessary next steps.

NOTEBOOK EVIDENCE

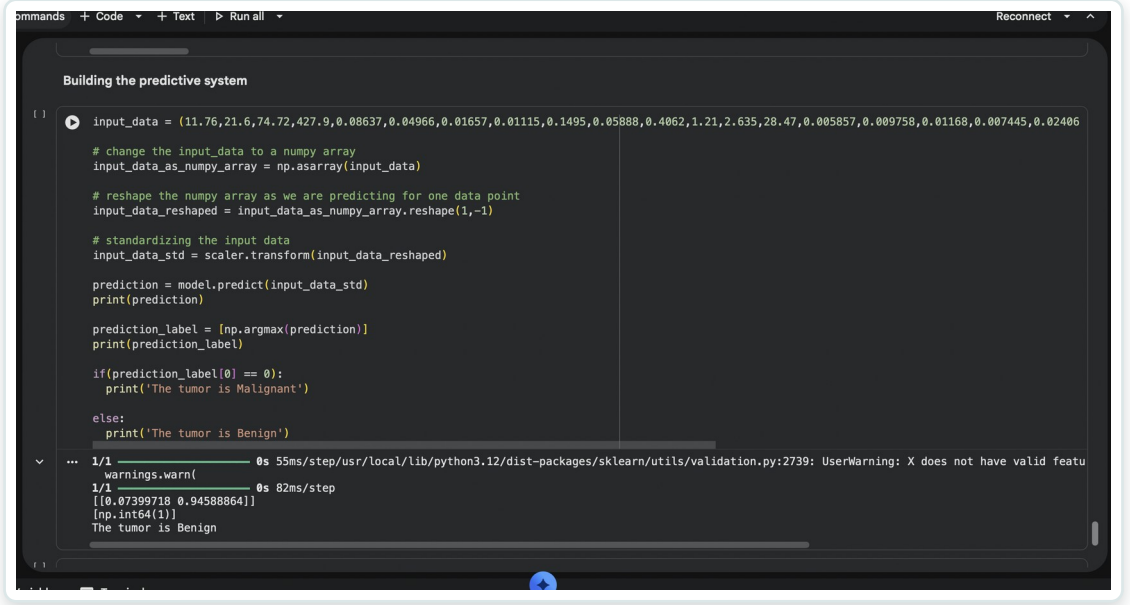
Evaluation and prediction, straight from the notebook

The test-set evaluation call reporting 96.49% accuracy, and the prediction system correctly classifying a new, unseen patient measurement as benign.



```
loss, accuracy = model.evaluate(X_test_std, Y_test)
print(accuracy)

4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 27ms/step - accuracy: 0.9649 - loss: 0.1059
0.9649122953414917

print(X_test_std.shape)
print(X_test_std[0])

(114, 30)
[-0.04462793 -1.41612656 -0.05903514 -0.16234067  2.0202457  -0.11323672
  0.18500609  0.47102419  0.63336386  0.26335737  0.53209124  2.62763999
  0.62351167  0.11405261  1.01246781  0.41126289  0.63848593  2.88971815
 -0.41675911  0.74270853 -0.32983699 -1.67435595 -0.36854552 -0.38767294
  0.32655007 -0.74858917 -0.54689089 -0.18278004 -1.23064515 -0.6268286 ]

Y_pred = model.predict(X_test_std)

4/4 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step

print(Y_pred.shape)
print(Y_pred[0])

(114, 2)
[0.28238547 0.7927458 ]

print(X_test_std)

[[-0.04462793 -1.41612656 -0.05903514 ... -0.18278004 -1.23064515
  -0.6268286 ]
 [ 0.24583601 -0.06219797  0.21802678 ...  0.54129749  0.11047691
```

Fig. — model.evaluate() on the 114 test samples: 96.49% accuracy



```
ommands + Code + Text ▷ Run all                                   Reconnect

Building the predictive system

input_data = (11.76,21.6,74.72,427.9,0.08637,0.04966,0.01657,0.01115,0.1495,0.05888,0.4062,1.21,2.635,28.47,0.005857,0.009758,0.01168,0.007445,0.02406

# change the input_data to a numpy array
input_data_as_numpy_array = np.asarray(input_data)

# reshape the numpy array as we are predicting for one data point
input_data_reshaped = input_data_as_numpy_array.reshape(1,-1)

# standardizing the input data
input_data_std = scaler.transform(input_data_reshaped)

prediction = model.predict(input_data_std)
print(prediction)

prediction_label = [np.argmax(prediction)]
print(prediction_label)

if(prediction_label[0] == 0):
  print('The tumor is Malignant')

else:
  print('The tumor is Benign')

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 55ms/step/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid featu
  warnings.warn(
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 82ms/step
[[0.07399718 0.94588864]]
[np.int64(1)]
The tumor is Benign
```

Fig. — prediction system classifying a new patient as benign

APPLICATIONS & LIMITATIONS

What it could support — and what stands in the way

POTENTIAL APPLICATIONS

- Decision support in oncology clinics, alongside pathologist review
- Triage in diagnostic labs, flagging likely-malignant cases for priority review
- Teaching and training on quantitative diagnostic features
- Research: rapid analysis of tumour characteristics at scale

Important qualification: this is a proof of concept validated on one historical dataset. It is not a diagnostic device — real deployment would require independent validation, prospective clinical evaluation, and regulatory clearance.

LIMITATIONS

- Small dataset (569 samples, 114 test) — accuracy estimates carry real uncertainty
- Data dates from the early 1990s and one institution only
- Only accuracy and loss reported; no sensitivity, specificity, or confusion matrix
- Binary only — no subtype or stage identification
- No interpretability — the network gives no explanation for individual predictions

CONCLUSION

A sound proof of concept, honestly evaluated

The network reached 96.49% test accuracy, exceeding the 90% target, with a narrow train/test gap and steadily declining validation loss — clear signs of genuine generalisation rather than overfitting. Careful preprocessing — checking for missing values, comparing class-wise feature means, and fitting the scaler on training data only — accounted for much of the final performance, arguably more than the network architecture itself.

Within its bounds — a small, single-source, historical dataset evaluated on accuracy alone — the project supports a clear conclusion: neural networks are well suited to structured medical classification problems, and with careful preprocessing and honest evaluation they can achieve strong, reproducible results.

NEXT STEPS

- Report confusion matrix, sensitivity & specificity
- Apply k-fold cross-validation for a reliable accuracy estimate
- Validate on independent, contemporary, multi-institution data
- Apply SHAP for prediction-level interpretability
- Explore ensembles with tree-based classifiers
- Extend to multi-class subtyping and multi-modal (imaging) input

Thank You

Vaidehi Aggarwal